CS5730 Operating Systems: 2013 Midterm Solutions

2013 · CS5730 Operating Systems Midterm (2013/11/11)

1. Short Answer: Your answers should be as concise as possible; aim for three or four sentences.

Q1.1 (6 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs. 2013

ANSWER

Disabling interrupts works on a uniprocessor only for **trusted kernel code**. It is the wrong tool for **user-level** synchronization because:

1. **It is a privileged instruction.** Letting user programs execute it breaks dual-mode protection; the hardware traps if user mode tries.
2. **The OS loses control of the CPU.** While interrupts are off, the **timer interrupt** cannot fire, so no preemption or context switch can happen. A buggy or malicious program could disable interrupts and never re-enable them (e.g. an infinite loop in its critical section). It would then **monopolize the CPU** and hang the whole system.
3. **Other events are delayed or lost.** I/O-completion interrupts and clock ticks are postponed, so devices sit idle, the system clock drifts and response time suffers. User critical sections can be long, which makes this worse.

(It also does not scale: on a multiprocessor it only stops the local CPU.) User programs should instead use OS-provided semaphores/mutexes or atomic hardware instructions such as test_and_set or swap.

Q1.2 (6 pt) What is busy-waiting in the discussion of critical section problem? Explain how busy-waiting may degrade the system performance, or even deadlock? 2013

ANSWER

Busy waiting (spinning) means a process waiting to enter its critical section repeatedly executes a loop that tests a condition, e.g. `while (test_and_set(&lock));` or `while (S <= 0);`. It never gives up the CPU while waiting.

Performance: the spinning process **wastes CPU cycles** that other processes could use. On a uniprocessor this is even worse: the spinner burns its whole time slice, and the lock holder cannot run during that time to finish and release the lock. So waiting gets longer and throughput drops.

Deadlock: with priority scheduling, suppose a high-priority process H spins on a lock held by a low-priority process L. H is always runnable, so the scheduler never runs L. L can never release the lock, and H spins forever. Both are stuck (a priority-inversion deadlock).

Fix: a blocking semaphore, where a waiting process sleeps on a queue instead of spinning.

Q1.3 (6 pt) In class, we discuss the bounded buffer problem which uses two semaphore, empty and full. Explain the usage of these two semaphores. 2013

ANSWER

Three semaphores protect a buffer of *n* slots: **mutex = 1** (mutual exclusion on the buffer), **empty = n** and **full = 0**.

- **empty** counts **empty slots**. The producer calls `wait(empty)` before inserting, so it blocks when the buffer is full. The consumer calls `signal(empty)` after removing an item.
- **full** counts **filled slots**. The consumer calls `wait(full)` before removing, so it blocks when the buffer is empty. The producer calls `signal(full)` after inserting.

```
Producer:                          Consumer:
do {                               do {
    /* produce item */                 wait(full);
    wait(empty);                       wait(mutex);
    wait(mutex);                       /* remove item */
    /* add item to buffer */           signal(mutex);
    signal(mutex);                     signal(empty);
    signal(full);                      /* consume item */
} while (true);                    } while (true);
```

Together they replace busy-wait loops, so all n slots can be used. Order matters: calling wait(mutex) before wait(empty) can deadlock.

Q1.4 (5 pt) Illustrate how the two process scheduling criteria, CPU utilization and throughput, may conflict with each other? 2013

ANSWER

CPU utilization measures how busy the CPU is; throughput measures how many processes **complete** per unit time. Maximizing one can hurt the other:

- Running **long CPU-bound jobs** keeps the CPU near 100% busy (high utilization). But each job takes a long time, so **few jobs finish** per unit time (low throughput). Short jobs stuck behind them also delay completion (convoy effect).
- Favoring **many short jobs** (SJF, short quantum) completes more jobs per unit time (high throughput). But it causes more context switches, whose overhead is not useful work, and short I/O-bound jobs leave the CPU idle while they wait on I/O. So utilization drops.

Q1.5 (6 pt) What happens if one thread of a multi-threaded process call `fork()`? What happens if one thread of a multi-threaded process call `exec()`? 2013

ANSWER

fork(): this depends on the system. Some UNIX systems provide **two versions** of fork(). One **duplicates all threads** of the process. The other (the POSIX behaviour) **duplicates only the thread that called fork()**, so the child is single-threaded. If exec() is called right after fork(), duplicating only the calling thread is enough, because exec replaces everything anyway. If the child does not call exec, duplicating all threads may be needed.

exec(): exec works as usual. The program named in its parameter **replaces the entire process, including all of its threads**. The other threads are destroyed, and the new program starts with a single thread.

Q2 (10 pt) A variation of the round-robin scheduler is the regressive round-robin scheduler. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process (CPU-bound or I/O-bound) does the regressive round-robin scheduler favor? Explain. 2013

ANSWER

It favors CPU-bound processes.

- A process that **uses its entire quantum** (does not block for I/O) is CPU-bound behaviour. It is **rewarded**: +10 ms of quantum, up to 100 ms, **and** a priority boost.
- A process that **blocks before using its quantum** is I/O-bound / interactive behaviour. It is **penalized**: −5 ms of quantum, and its priority is **not** raised.

So over time CPU-bound (batch) processes get longer time slices and higher priority, and I/O-bound processes get shorter slices and stay at lower priority. This is the opposite of the usual goal of favoring interactive processes.

Q3 (10 pt) Can a multi-threaded solution using multiple user-level threads achieve better performance on a multiprocessor system, which uses the many-to-one threading model, than on a single processor system? Explain. 2013

ANSWER

No, not significantly. In the many-to-one model, all user-level threads of the process are mapped onto **one kernel thread**. The kernel schedules only kernel threads, so it sees the process as a single schedulable entity. It can run on only **one processor at a time**, which means only one user thread executes at any moment even if other CPUs are idle. The program gets **no parallelism**, and its performance is essentially the same as on a single processor. Also, if any thread makes a blocking system call, the whole process (all threads) blocks.

(The only gain is indirect: other processes can use the other CPUs, so the program may face less competition.) To benefit from multiple processors you need one-to-one or many-to-many threading.

4. Multilevel queue scheduling

Q4.1 (4 pt) List two possible scheduling among queues, which we discussed in class. 2013

ANSWER

Two ways to schedule **among** the queues of a multilevel queue:

1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a higher queue preempts a lower one. Lower queues may **starve**.
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

Q4.2 (5 pt) Why is it important for the scheduler to distinguish I/O-bound programs from CPU-bound programs? 2013

ANSWER

I/O-bound and CPU-bound processes behave very differently. **I/O-bound** processes have many short CPU bursts. **CPU-bound** processes have few, long bursts.

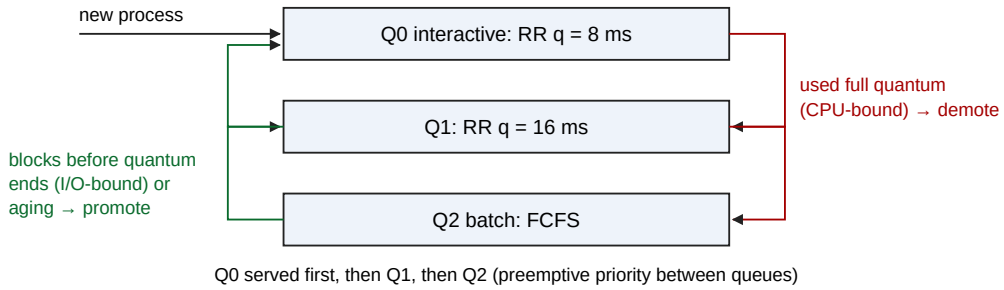
- If an I/O-bound process gets the CPU quickly, it issues its next I/O request soon and then gets out of the way. The **devices stay busy** while CPU-bound processes use the CPU, so **both CPU and I/O utilization** improve.
- I/O-bound processes are usually **interactive**, so serving them first gives good **response time**.
- If they wait behind long CPU bursts (convoy effect), devices go idle and users see poor response.

So the scheduler should give I/O-bound processes higher priority and short quanta, and give CPU-bound processes lower priority with longer quanta (fewer switches).

Q4.3 (6 pt) Modify multilevel queue scheduling so it can distinguish I/O-bound programs from CPU-bound programs. 2013

ANSWER

Change it into a **multilevel feedback queue**, where a process can **move between queues** based on its observed CPU bursts:



- A new process enters **Q0** (small quantum). If it **uses the full quantum**, it is behaving CPU-bound and is **demoted** to Q1, then to Q2.
- If it **blocks for I/O before its quantum expires**, it is I/O-bound and stays in, or is **promoted** to, a higher queue.
- **Aging:** a process that waits too long in a low queue is moved up, which prevents starvation.

So I/O-bound processes end up in high-priority, short-quantum queues and CPU-bound processes in low-priority, long-quantum queues. Parameters: number of queues, algorithm per queue, promotion rule, demotion rule, entry queue.

5.

Q5.1 (5 pt) Write to demonstrate how to use the atomic instruction TestAndSet for critical section problem. 2013

ANSWER

```
boolean test_and_set(boolean *target) {   /* executed ATOMICALLY by hardware */
    boolean rv = *target;
    *target = true;
    return rv;
}

boolean lock = false;                     /* shared */

do {
    while (test_and_set(&lock))
        ;                                 /* spin while lock was already true */
    /* critical section */
    lock = false;                         /* release */
    /* remainder section */
} while (true);
```

The first process to call test_and_set gets back **false** (the lock was free) and has atomically set lock to true, so it enters. Every other caller gets **true** and keeps looping until the holder sets lock = false. This gives mutual exclusion. (Bounded waiting needs an extra `waiting[]` array.)

Q5.2 (5 pt) Illustrate how your solution will fail if TestAndSet is not atomic. 2013

ANSWER

If test_and_set is not atomic, its read (`rv = *target`) and write (`*target = true`) can be interleaved. Start with lock = false:

Step	P0	P1	lock
1	rv = lock → false		false
2	*preempted*	rv = lock → false	false
3		lock = true; returns false → **enters CS**	true
4	lock = true; returns false → **enters CS**		true

Both processes saw the lock as free, so **both are in the critical section: mutual exclusion is violated.**

Q5.3 (5 pt) Write to demonstrate how to use the atomic instruction Swap for critical section problem. 2013

ANSWER

```
/* swap: executed ATOMICALLY by hardware */
void swap(boolean *a, boolean *b) {
    boolean temp = *a;
    *a = *b;
    *b = temp;
}

boolean lock = false;           /* shared */
boolean key;                    /* local to each process */
```

```
do {
    key = true;
    while (key == true)
        swap(&lock, &key);  /* key=old lock; lock=true */
    /* critical section */
    lock = false;
    /* remainder section */
} while (true);
```

A process loops until a swap returns **key = false**, meaning the lock was free. That same swap atomically set lock = true, so only one process gets out of the loop. The others keep getting key = true until lock is reset to false.

Q5.4 (5 pt) Illustrate how your solution will fail if Swap is not atomic. 2013

ANSWER

If swap is not atomic, the three steps (`temp=*a; *a=*b; *b=temp`) of two processes can interleave. Start with lock = false, and each process has key = true:

Step	P0	P1	lock
1	temp0 = lock → false		false
2	*preempted*	temp1 = lock → false	false
3		lock = key1 (true); key1 = temp1 = false → **enters CS**	true
4	lock = key0 (true); key0 = temp0 = false → **enters CS**		true

Both read the old value false before either wrote true, so both leave the loop. **Mutual exclusion is violated.**

Q6 (10 pt) Race conditions are possible in many computer systems. Consider a banking system with two methods: `deposit(amount)` and `withdraw(amount)`. These two methods are passed the amount that is to be deposited or withdrawn from a bank account. Assume that a husband and wife share a bank account and that concurrently the husband calls the `withdraw()` method and the wife calls `deposit()`. 2013

(The question stops here on the paper. The textbook version asks: describe how a race condition is possible and what might be done to prevent it.)

ANSWER

deposit() does `balance = balance + amount` and withdraw() does `balance = balance - amount`. At machine level each is **load balance → modify in a register → store balance**, which is not atomic. Suppose balance = 100, the husband withdraws 50 and the wife deposits 50 (the correct result is 100):

Step	Husband: withdraw(50)	Wife: deposit(50)	balance
1	reg1 = balance (100)		100
2		reg2 = balance (100)	100
3	reg1 = 100 − 50; balance = 50		50
4		reg2 = 100 + 50; balance = 150	**150**

The withdrawal is lost, and the result would be 50 if steps 3 and 4 were swapped. The outcome depends on the order of execution, which is a **race condition**.

Prevention: make the update a critical section, so only one method can touch balance at a time. Use a lock or semaphore (`wait(mutex); balance ±= amount; signal(mutex);`), a monitor, or synchronized methods.

Q7 (6 pt) Consider the following code segment: 2013

```
pid_t pid;

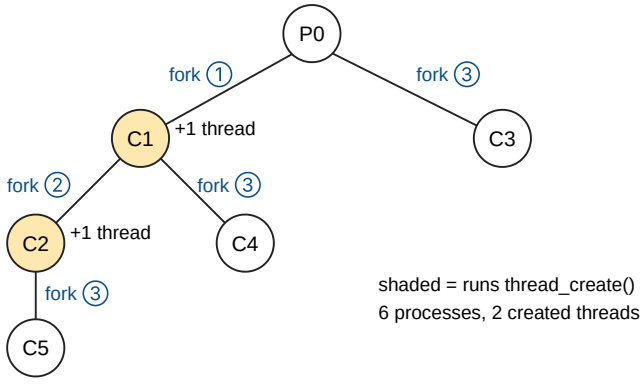
pid = fork();
if (pid == 0) { /* child process */
    fork();
    thread_create( . . .);
}
fork();
```

a) How many unique processes are created?
b) How many unique threads are created?

ANSWER

Trace each fork (the original process is P0):

- fork ① gives P0 and child C1. Only C1 has pid == 0.
- Inside the if, C1 calls fork ②, creating C2. **C1 and C2 each call thread_create()**, so 2 threads are created.
- The final fork ③ is executed by P0, C1 and C2, creating C3, C4 and C5.



(a) 6 unique processes (P0, C1–C5; 5 newly created if the original is not counted).
(b) 2 unique threads created by thread_create(), one in C1 and one in C2. The last fork() duplicates only the calling thread (POSIX), so it creates no extra threads.